

Automated Source Code Review

This Source Code Review report is submitted to Application Security Assessment of a source code bundle named “**cr-master**” by

Niyaz Mursid Dolon

Executive Summary

This report presents the findings of an automated and manual source code security review conducted on the "cr-master" application bundle, a PHP-based web application. The analysis was performed using Snyk integrated with GitHub repository hosting and Visual Studio Code with the Snyk Security Extension for real-time local analysis. Manual source code review was additionally conducted to validate critical findings.

A total of 9 security issues were identified during automated scanning, of which 5 significant findings were investigated in detail and are presented in this report. The findings by severity are as follows: 2 High-severity, 2 Medium-severity, and 1 Low-severity issue.

Finding 01 (High — CWE-79): A reflected Cross-Site Scripting (XSS) vulnerability was identified in `index.php` at lines 32 and 34, where the application reflects the unsanitized `$_SERVER['PHP_SELF']` server variable directly into the HTML response. This allows an attacker to inject and execute arbitrary JavaScript in the victim's browser, enabling session hijacking, credential theft, malicious redirection, and unauthorized user actions.

Finding 02 (Medium — CWE-798/CWE-259): Database credentials were hardcoded directly in `classes/db.php` at line 2, where the MySQL connection string contains plaintext username and password ("pentesterlab"/"pentesterlab"). Exposure of the source code through version control history or unauthorized access would directly compromise database confidentiality and integrity.

Finding 03 (Low — CWE-1004): An authentication cookie named "auth" is set in `login.php` (lines 7–8) without the `HttpOnly` flag, allowing client-side JavaScript to access the cookie. This vulnerability compounds the XSS risk identified in Finding 01, as a successful XSS attack could directly steal the session cookie and enable session hijacking.

Finding 04 (Medium — CWE-327): The application uses MD5 for password hashing in `classes/user.php` (lines 48–55). MD5 is cryptographically broken and susceptible to brute-force and rainbow table attacks, enabling an attacker to recover plaintext passwords from the database if it is compromised.

Finding 05 (High — CWE-434): An unrestricted file upload vulnerability was identified in `classes/user.php` (lines 15–21). The validation logic only checks whether the filename contains ".pdf" using a weak regex pattern, which can be bypassed using crafted filenames such as "malicious.php.pdf" or double extensions. Successful exploitation could allow an attacker to upload and execute arbitrary server-side code, resulting in Remote Code Execution (RCE) and full application compromise.

It is recommended that all findings be remediated in order of severity, with immediate priority given to the XSS vulnerability and the unrestricted file upload issue. Regular automated scanning should be incorporated into the development workflow to prevent future security regressions.

Contents

Chapter 1 — Objective

Chapter 2 — Methodology

Tools & Utility

Step of Analysis

Chapter 3 — Work Details

Chapter 4 — Conclusion

Chapter 1: Objective

The objective of this report is to perform an automated source code review of the "cr-master" source code bundle to identify security vulnerabilities and coding weaknesses. The analysis focuses on detecting issues such as injection flaws, insecure cookie handling, hardcoded credentials using industry-standard tools, etc. This review aims to provide a structured findings report with severity classification and remediation recommendations, following best practices in application security assessment.

Chapter 2: Methodology

Tools & Utility

1. Git CMD — for uploading source code to GitHub via CLI
2. GitHub (<https://github.com/>) — repository hosting and version control
3. Snyk (<https://app.snyk.io/>) — automated vulnerability scanning
4. Visual Studio Code — local code analysis with Snyk Security Extension

Steps of Analysis

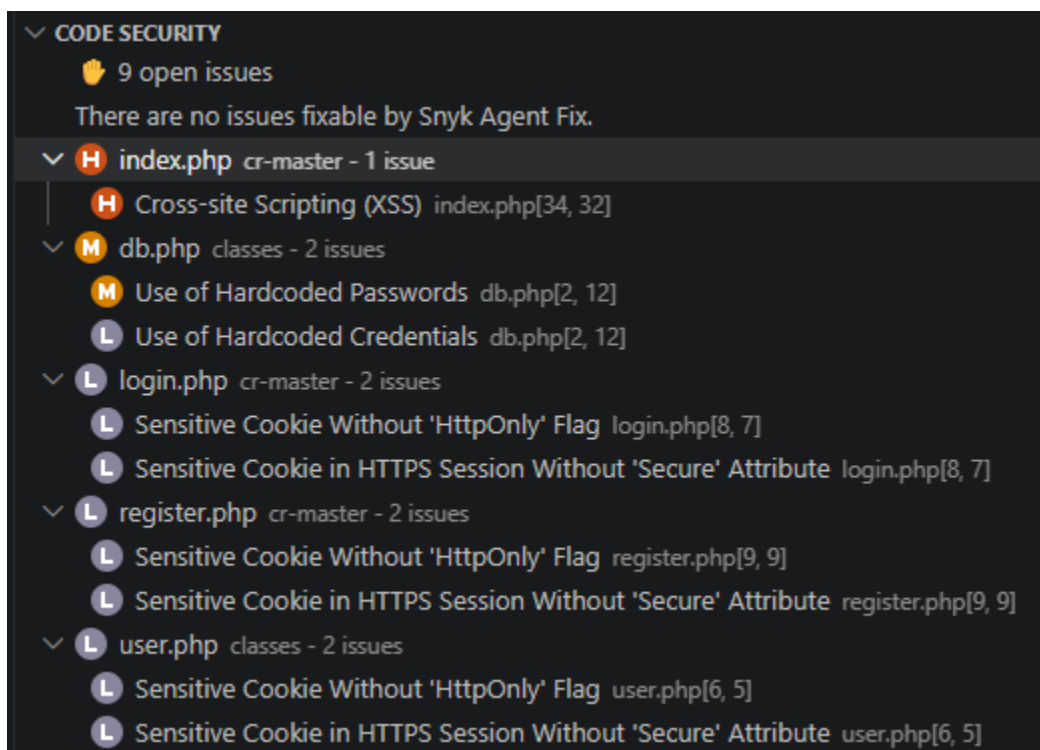
1. The target source code bundle "cr-master" was securely collected and prepared for analysis.
2. The source code was uploaded to a GitHub repository using Git CLI commands.
3. The GitHub repository was integrated with Snyk to enable automated security scanning and vulnerability detection.
4. The Snyk Security Extension was installed and connected within Visual Studio Code for local real-time analysis.
5. Automated scanning was performed to identify vulnerabilities, insecure configurations, and coding weaknesses.
6. Manual source code review was additionally performed using Visual Studio Code to identify further security issues and validate critical findings.
7. Identified findings were reviewed and categorized by severity level and CWE reference.
8. Supporting screenshots were captured as evidence for this report.

Chapter 3: Work Details

A total of 9 security issues were identified during the automated source code review of the “cr-master” application bundle. Additional manual source code review was also conducted to validate critical findings and identify further security weaknesses. Detailed descriptions, risk impacts, and remediation recommendations for the major findings are provided in the following sections.

Summary of Important Findings:

- Finding 01 — Cross-Site Scripting (XSS) — High
- Finding 02 — Hardcoded Database Credentials — Medium
- Finding 03 — Sensitive Cookie Without HttpOnly Flag — Low
- Finding 04 — Weak Password Hashing Mechanism (MD5) — Medium
- Finding 05 — Potential Remote Code Execution via Unrestricted File Upload— High



Finding 01 — Cross-Site Scripting (XSS)

Severity: High

CWE Reference: CWE-79 — Improper Neutralization of Input During Web Page Generation

Affected File: index.php

Affected Line: Line 32 & 34

Description

The application reflects user-controlled server variables into the HTML response without sufficient output encoding or sanitization. This creates a reflected Cross-Site Scripting (XSS) vulnerability.

Vulnerable Code:

```
<form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post">
```

Risk Impact

- Session hijacking
- Credential theft
- Browser-side malicious script execution
- Malicious redirection
- Unauthorized user actions

Recommendation

- Validate and sanitize all user-supplied and server-supplied input before rendering.
- Apply output encoding using `htmlspecialchars()` or `htmlspecialchars()`:
- Avoid directly reflecting server variables (e.g. `$_SERVER['PHP_SELF']`) into HTML.
- Implement a Content Security Policy (CSP) header.

Secure implementation example:

```
<?php echo htmlentities($_SERVER['PHP_SELF'], ENT_QUOTES, 'UTF-8'); ?>
```

The screenshot displays a code editor on the left and a security tool analysis on the right. The code editor shows the following PHP code in `index.php`:

```
1 <?php
15 if(isset($_POST["submit"])) {
17 }
18 require "header.php";
19 ?>
20
21 <div class="row">
22 <div class="col-lg-12">
23 <?php if (isset($error)) { ?>
24 <span class="text text-danger"><b><?php echo $error; ?></b></span>
25 <?php ?>
26 <h1>Hi <?php echo h($user); ?></h1>
27 <h3>Your files</h3>
28 <ul>
29 <?php foreach (User::getfiles($user) as $file) { ?>
30 <li><a href="/files/<?php echo h($user); ?>/><?php echo h($file); ?></a>
31 <?php ?>
32 </ul>
33 <h3>Upload (only PDF)</h3>
34 <form action="<?php echo $_SERVER['PHP_SELF']; ?>" method="post" enctype="multipart/form-data">
35 <input type="file" name="file" id="file">
```

The security tool analysis on the right identifies a **Cross-site Scripting (XSS)** issue. It notes that unsanitized input from an HTTP header flows into the echo statement, which is used to render an HTML page. The analysis includes a **FIXED CODE EXAMPLES** section showing the corrected code:

```
- <title><?php echo $_SERVER["HTTP_HOST"] ?> :: Pimcore</title>
+ <title><?php echo htmlentities($_SERVER["HTTP_HOST"], ENT_QUOTES, 'UTF-8') ?> :: Pimcore</title>
```

Finding 02 — Hardcoded Database Credentials

Severity: Medium

CWE Reference: CWE-798 — Use of Hard-coded Credentials / CWE-259 — Use of Hard-coded Password

Affected File: classes/db.php

Affected Line: Line 2

Description

Database credentials (username and password) were hardcoded directly within the application source code. This exposes sensitive authentication information to anyone with access to the codebase.

Vulnerable Code:

```
$lnk = mysql_connect("127.0.0.1", "pentesterlab", "pentesterlab");
```

Risk Impact

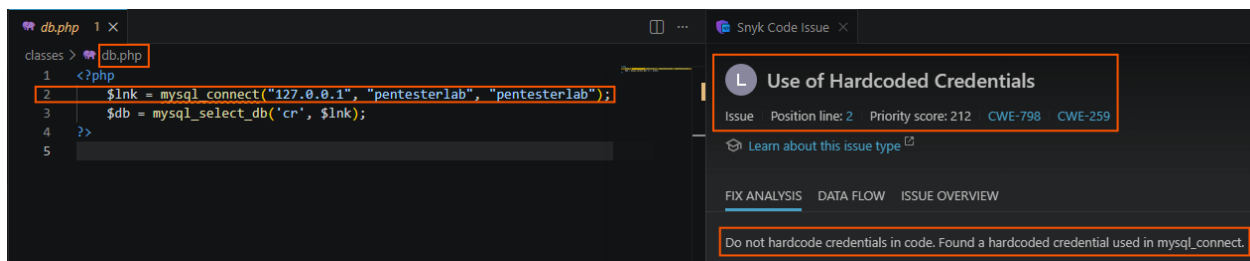
- Unauthorized database access if source code is exposed
- Credential disclosure through version control history
- Privilege escalation
- Lateral movement within infrastructure

Recommendation

- Remove hardcoded credentials from source code immediately.
- Store credentials using environment variables or a secure secrets manager.
- Apply the principle of least privilege for all database accounts.
- Rotate any exposed credentials without delay.

Secure implementation example:

```
$db_pass = getenv("DB_PASSWORD");
```



Finding 03 — Sensitive Cookie Without HttpOnly Flag

Severity: Low

CWE Reference: CWE-1004 — Sensitive Cookie Without 'HttpOnly' Flag

Affected File: login.php

Affected Line: Line 7 & 8

Description

The application sets an authentication cookie ("auth") without enabling the HttpOnly attribute. This allows the cookie to be accessed by client-side JavaScript, increasing the risk of theft via XSS attacks.

Vulnerable Code:

```
setcookie("auth", User::createcookie($_POST['username'], $_POST['password']));
```

Risk Impact

- Cookie theft via JavaScript (especially in combination with XSS)
- Session hijacking
- Unauthorized session reuse

Recommendation

- Always set the HttpOnly attribute on authentication and session cookies.
- Use the full setcookie() signature including security flags:

Secure implementation example:

```
setcookie("auth", $value, time()+3600, "/", null, true, true); // Secure=true, HttpOnly=true
```

The screenshot displays a code editor on the left and a vulnerability analysis panel on the right. In the code editor, the file `login.php` is open, showing PHP code. Lines 7 and 8 are highlighted with a red box: `if (User::login($_POST['username'], $_POST['password'])) { setcookie("auth", User::createcookie($_POST['username'], $_POST['password'])); header('Location: /index.php'); die(); } else { $error = "Invalid credentials"; }`. The right panel shows an issue titled "Sensitive Cookie Without 'HttpOnly' Flag" with a position line of 8 and a priority score of 434. The description states: "setcookie misses the HttpOnly attribute (it is false by default). Set it to true to protect the cookie from possible malicious code on client side." Under the "FIXED CODE EXAMPLES" section, it shows the corrected code: `setcookie('feAutologin', '', time() - 3600, '/', null, true, true);`.

Finding 04 — Weak Password Hashing Mechanism (MD5)

Severity: Medium

CWE Reference: CWE-327 — Use of a Broken or Risky Cryptographic Algorithm

Affected File: classes/user.php

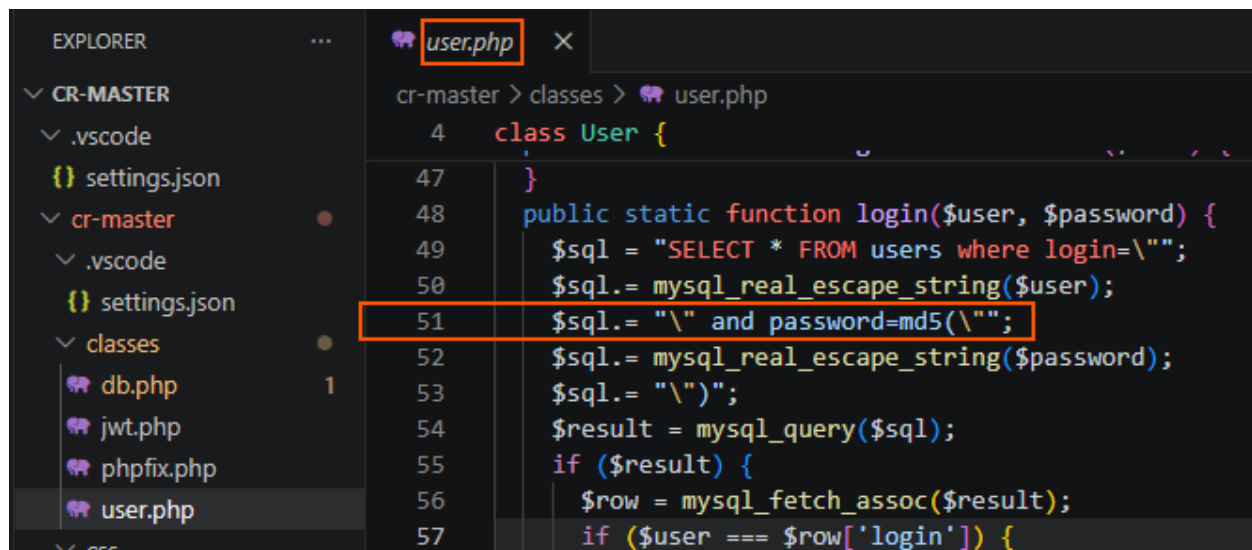
Affected Line: Line 48 to 55

Description

The application uses the MD5 hashing algorithm for password verification during user authentication.

vulnerable code:

```
public static function login($user, $password) {  
    $sql = "SELECT * FROM users where login=\"\"";  
    $sql.= mysql_real_escape_string($user);  
    $sql.= "\" and password=md5(\"\"";  
    $sql.= mysql_real_escape_string($password);  
    $sql.= "\"\"";  
    $result = mysql_query($sql);  
    if ($result) {
```



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'CR-MASTER', '.vscode', and 'classes'. The file 'user.php' is selected in the 'classes' folder. The code editor shows the following code:

```
4  class User {  
47 }  
48 public static function login($user, $password) {  
49     $sql = "SELECT * FROM users where login=\"\"";  
50     $sql.= mysql_real_escape_string($user);  
51     $sql.= "\" and password=md5(\"\"";  
52     $sql.= mysql_real_escape_string($password);  
53     $sql.= "\"\"";  
54     $result = mysql_query($sql);  
55     if ($result) {  
56         $row = mysql_fetch_assoc($result);  
57         if ($user === $row['login']) {
```

Line 51 is highlighted with a red box.

MD5 is considered cryptographically weak and vulnerable to collision and brute-force attacks. Modern security standards no longer recommend MD5 for password storage or authentication purposes.

Risk Impact

- Crack user passwords using brute-force or rainbow table attacks

- Gain unauthorized access to user accounts
- Reuse compromised credentials across systems
- Escalate attacks if passwords are reused elsewhere

Recommendation

- Avoid using MD5 for password hashing.
- Use modern password hashing algorithms such as:
 - bcrypt
 - Argon2
 - PBKDF2
- Implement salted password hashing.
- Use PHP built-in secure password handling functions.

Secure Implementation Example

```
// Store password securely
$hashedPassword = password_hash($password, PASSWORD_BCRYPT);

// Verify password securely
if (password_verify($password, $hashedPassword)) {
    echo "Login successful";
}
```

Finding 05 — Potential Remote Code Execution via Unrestricted File Upload

Severity: High

CWE Reference: CWE-434 — Unrestricted Upload of File with Dangerous Type

Affected File: classes/user.php

Affected Line: Line 15 to 21

Description

The application allows file uploads with insufficient validation controls. The validation logic only checks whether the filename contains “.pdf”, which may be bypassed using crafted filenames or double extensions.

Vulnerable code:

```
public static function addfile($user) {  
    $file = "files/".$user."/".basename($_FILES["file"]["name"]);  
    if (!preg_match("/\.(pdf)/", $file)) {  
        return "Only PDF are allowed";  
    } elseif (!move_uploaded_file($_FILES["file"]["tmp_name"], $file)) {  
        return "Sorry, there was an error uploading your file.";  
    }  
}
```

Risk Impact

- Upload malicious executable files
- Execute arbitrary code on the server
- Compromise the application environment
- Gain unauthorized system access
- Perform Remote Code Execution (RCE)

Recommendation

- Strictly validate file extensions and MIME types.
- Rename uploaded files using random generated names.
- Store uploaded files outside the web root directory.
- Block execution permissions in upload directories.
- Implement allowlist-based file validation.

Secure Implementation Example

```
$allowedExtensions = ['pdf'];
```

```
$fileName = $_FILES['file']['name'];
```

```
$fileExtension = strtolower(pathinfo($fileName, PATHINFO_EXTENSION));
```

```

if (!in_array($fileExtension, $allowedExtensions)) {
    die("Only PDF files are allowed.");
}

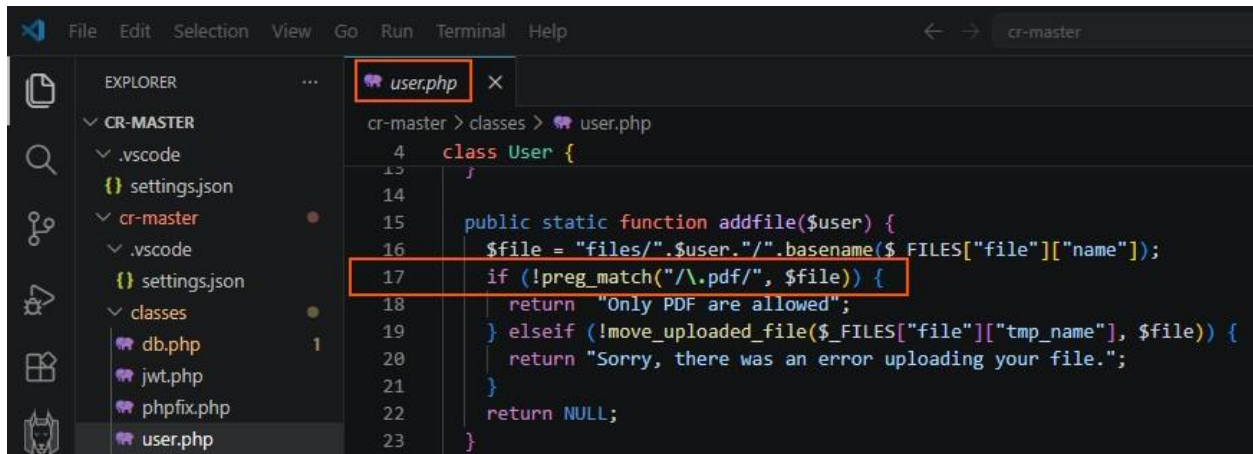
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $_FILES['file']['tmp_name']);

if ($mime !== 'application/pdf') {
    die("Invalid file type.");
}

$newFileName = bin2hex(random_bytes(16)) . '.pdf';

move_uploaded_file(
    $_FILES['file']['tmp_name'],
    '/secure_uploads/' . $newFileName
);

```



```

cr-master > classes > user.php
4  class User {
13  }
14
15  public static function addfile($user) {
16      $file = "files/" . $user . "/" . basename($_FILES["file"]["name"]);
17      if (!preg_match("/\.pdf/", $file)) {
18          return "Only PDF are allowed";
19      } elseif (!move_uploaded_file($_FILES["file"]["tmp_name"], $file)) {
20          return "Sorry, there was an error uploading your file.";
21      }
22      return NULL;
23  }

```

Chapter 4: Conclusion

The automated source code review of the "cr-master" bundle successfully identified 5 security issues: 1 High severity (Cross-Site Scripting), 1 Medium severity (Hardcoded Database Credentials), and 3 Low severity issues related to insecure cookie handling. The integration of Snyk with GitHub and Visual Studio Code enabled efficient and systematic detection of these vulnerabilities. It is recommended that all findings be remediated in order of severity, with immediate priority given to the XSS and hardcoded credential issues, as these present the greatest risk to application integrity and data confidentiality. Regular automated scanning should be incorporated into the development workflow to prevent future security regressions.